TECHNICAL PROJECT GUIDE

PosterScore AI

How the application was built, which technologies it uses, and how the poster analysis logic works.

UPLOAD	MEASURE	ANALYZE	IMPROVE

PosterScore AI is a full-stack MVP that reviews a poster like a creative director. It identifies whether the upload is actually a poster, combines measured image facts with visual reasoning, calculates a type-aware score in backend code, and produces prioritized design fixes.

Local development

```
Frontend: http://localhost:5173/    Backend: http://localhost:8787    Start command: npm run dev
```

1. Technology stack and architecture

The project is intentionally small and direct: a React client handles interaction, while an Express server owns image validation, Gemini calls, quality control, and scoring.

Layer	Technology	Purpose
Frontend	React 19 + TypeScript	Upload flow, staged loading UI, report dashboard, responsive interactions.
Frontend tooling	Vite 6	Fast development server and production bundling.
Backend	Node.js + Express 5	API route, multipart upload handling, rate limiting, and provider errors.
AI	Google Gemini via @google/genai	Image understanding, poster classification, evidence checks, and recommendations.
Image processing	Sharp	Decode validation, rotation, resizing, JPEG preparation, and image measurements.
Validation	Zod	Strict analysis response validation and quality-control rules.
Testing	Vitest	Scoring, schema, correction-loop, and file-validation tests.

Project structure

src/App.tsx contains the main UI workflow. src/analysisApi.ts is the frontend API boundary. server/app.ts exposes the API. server/vision.ts calls Gemini. server/analysis.ts owns the scoring contract. server/validation.ts owns image safety and measurements.

The API key stays in the server-side .env file. It is never placed in a VITE_ variable and is never sent to the browser.

2. End-to-end analysis workflow

Step	Stage	Logic
01	Upload	The browser accepts JPG, JPEG, PNG, and WEBP files up to 10 MB. Drag-and-drop and file-picker flows use the same handler.
02	Validate again	Multer limits the request, then Sharp confirms the bytes are a decodable image. MIME type, extension, and size are checked.
03	Prepare and measure	The image is rotated, resized to a maximum of 2400px, flattened, and converted to JPEG. Width, height, aspect ratio, orientation, dominant colors, estimated whitespace, and coverage are measured.
04	Add context	Target platform, goal, audience, and brand information are included when supplied.
05	Ask Gemini	The prompt classifies poster versus non-poster, then requests seven categories with 2-4 evidence checks each.
06	Validate and correct	Zod validates the JSON. Quality checks reject contradictions, generic advice, duplicate recommendations, missing evidence, and identical category scores.
07	Calculate score	The backend averages evidence checks, applies poster-type weights, redistributes non-applicable branding weight, and assigns a rating label.
08	Render report	React shows confidence, limitations, the three-second test, category evidence, strengths, markers, improvements, and version history.

The visible loading stages mirror the real workflow so the user sees meaningful progress instead of an unexplained spinner.

3. Scoring logic

The model never owns the final score. It returns small evidence checks; the server is the source of truth for all numbers shown in the report.

Categories

Category	What it measures
Layout & hierarchy	Eye path, alignment, balance, spacing, and grouping.
Typography	Font choice, pairing, size contrast, hierarchy, and line breaks.
Readability & contrast	Text visibility, contrast clues, viewing distance, line length, and busy backgrounds.
Imagery / graphics	Relevance, resolution clues, cropping, and integration with the message.
Message & CTA	Message clarity, purpose, and visibility of the next action.
Color	Palette, contrast relationships, and consistent accent use.
Branding	Recognition and consistency when a logo or brand context makes it judgeable.

A category score is calculated as the average of its 0-10 checks multiplied by 10. The overall score is a weighted sum of category scores.

Example type-aware weights

Poster type	Highest emphasis
Promotional Offer	Layout 20%, Message & CTA 20%, Branding 15%
Event	Layout 20%, Typography 15%, Readability 15%, Imagery 15%
Restaurant/Food	Imagery 25%, Layout 15%, Readability 15%
Corporate	Typography 20%, Branding 20%, Readability 15%
Other types	Balanced base profile across all seven categories

Branding is conditional. When it cannot be judged, its weight becomes zero and is redistributed across the other categories instead of unfairly lowering the poster score.

4. User interface and report

The interface is organized around a simple three-stage product story: upload, diagnose, improve.

Screen	Main behavior
Landing / upload	Responsive hero layout, drag-and-drop zone, preview, file metadata, target platform, goal, audience, and brand context.
Loading	Poster preview, animated scan line, and eight named analysis stages.
Non-poster result	Explains that the upload is not a poster and asks for a designed poster instead of showing misleading scores.
Poster result	Score ring, rating, confidence, limitations, intended use, three-second attention order, measured image facts, and category breakdown.
Improvement detail	High/medium/low priority filters, numbered markers, problem, why it matters, and recommended fix.
Version history	Recent analyses are retained in browser localStorage for quick comparison.
Export	Browser print styles allow the report to be saved as a PDF.

Responsive design

+ Desktop uses a two-column hero and report layout.
+ Tablet collapses major grids while preserving readable spacing.
+ Mobile uses single-column sections, flexible file names, compact category rows, and full-width actions.
+ Print styles hide navigation and interactive controls for a clean report.

5. API, reliability, and security

API endpoints

Endpoint	Purpose
GET /api/health	Reports backend status, Gemini configuration, provider, and demo mode.
POST /api/analyze	Accepts multipart image data plus a JSON context field and returns the validated report.

Gemini reliability

+ Retries transient 503 and 429 provider failures.
+ Tries the configured model, then Gemini 3.7 Flash and Gemini 3.6 Flash fallbacks.
+ Uses JSON mode and validates the complete response with Zod on the server.
+ Runs up to two skeptical correction passes after quality-control failures.
+ Maps invalid-key, quota, model, busy-provider, and response-format failures to clearer UI messages.

Security and limits

+ The Gemini key is server-only.
+ Uploaded files are processed in memory and are not persisted by the backend.
+ The endpoint limits files to 10 MB and applies an in-memory limit of 12 analyses per 10 minutes per client.
+ There is no database, account system, or cloud report history in this MVP.

6. Testing, commands, and current limitations

Verification

The project includes automated tests for deterministic scoring, rating bands, poster-type weight behavior, conditional branding redistribution, malformed model responses, identical scores, correction passes, maximum correction attempts, MIME and extension validation, file-size limits, and corrupt image bytes.

Development commands

```
npm install npm run dev # frontend + backend npm run dev:ui # frontend only npm test npm run build
```

Environment modes

```
For UI-only testing without Gemini: VITE_USE_MOCK_ANALYSIS=true DEMO_MODE=true
```

Current limitations

+ Whitespace and coverage are image-level estimates, not OCR-based measurements.
+ Exact WCAG contrast ratios, font-size detection, and color-blind simulation are not yet implemented.
+ Arabic and RTL poster analysis is not yet specialized.
+ PDF export uses the browser print flow rather than a dedicated server PDF generator.
+ No persistent database, login, or cloud version history is included.

Summary

PosterScore AI is built so that Gemini supplies visual judgment and concrete evidence, while the application remains responsible for validation, scoring, reliability, and user experience. This separation keeps the MVP explainable, testable, and easier to improve as the analysis model evolves.